



CLAUDE.md & Claude Code Memory

How Claude Code remembers your project — context windows, CLAUDE.md hierarchy, imports, rules & AGENTS.md

📖 WHAT'S INSIDE

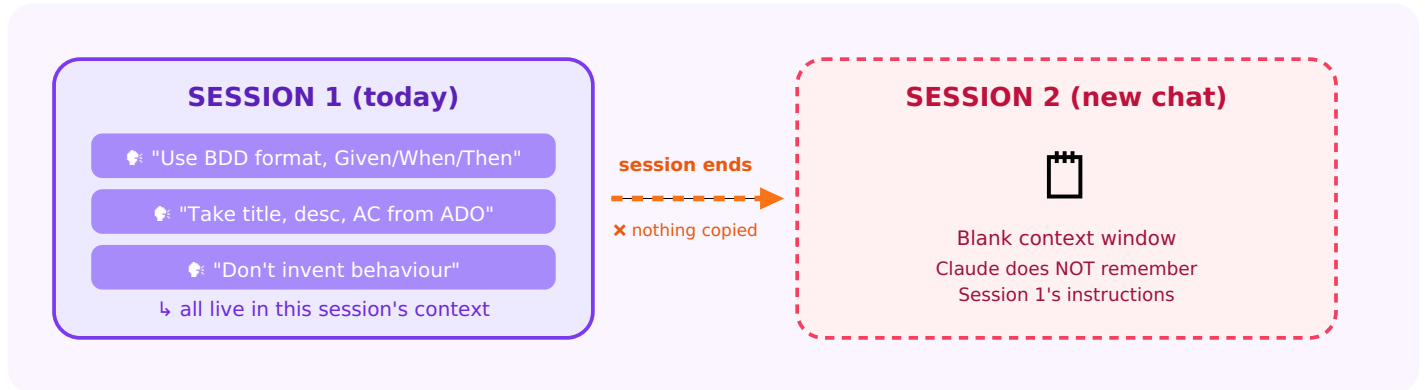
1. Why LLMs "forget" — sessions & context windows
2. What is CLAUDE.md and why it matters
3. Creating CLAUDE.md — `/init` vs manual
4. The 5 levels of CLAUDE.md (Org → Local)
5. Imports with `@` — and nested imports
6. AGENTS.md — symlink vs import
7. `.claude/rules/` — unconditional vs path-specific
8. Case study: AI Test Case Generator project
9. Cheat sheet & quick recap quiz

Why Does Claude "Forget" Everything?

Understanding sessions, context, and the context window

An LLM has **no permanent memory** between chat sessions. Every new Claude Code session starts with a completely **blank context window** — nothing from a previous conversation is automatically carried over, unless something outside the model (a file, a memory system, a retrieval step) explicitly feeds it back in.

Diagram — Session 1 vs Session 2



△ THE CORE PROBLEM

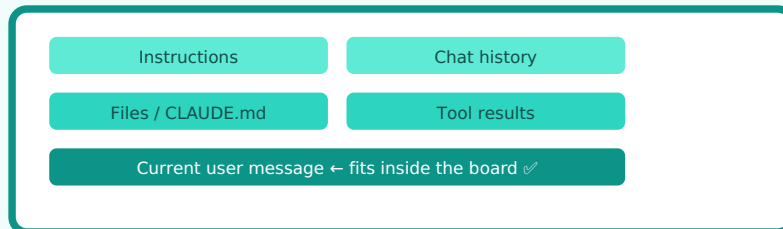
If you give Claude project instructions in one Claude Code session, don't assume they'll be available automatically in the next session. You'd have to repeat them every single time.

What exactly is "Context"?

Context = everything currently available to the model while generating a response: system/user instructions, conversation history, the current message, attached files, tool results, and anything the application chooses to include.

The Whiteboard Analogy

A context window = a whiteboard with fixed size



Whatever doesn't fit falls OUTSIDE the board
→ Claude can't use it ✗

New session = a brand-new, empty whiteboard. No previous context on it.

➡ TAKEAWAY

We need a reliable way to store **project-level instructions** outside the conversation itself — something Claude can reload every time. That's exactly what `CLAUDE.md` is for.

What is CLAUDE.md?

Problem → Solution, and the benefit of consistency

? PROBLEM

We don't want to repeat the same project-specific instructions every time we start a new Claude Code session.

✓ SOLUTION

`CLAUDE.md` stores those instructions at the project level. Claude Code loads them into the current session's context automatically.

i IMPORTANT NUANCE

CLAUDE.md does **not** give Claude permanent, human-like memory. Claude Code simply *reads/loads the file* and injects its contents into the current session's context — every time a session starts fresh.

📁 Typical rules you'd put in CLAUDE.md



Tooling

Always use Playwright for UI automation



Anti-pattern

Don't use hard-coded waits



Format

Follow a specific test-case / BDD format



Naming

Follow naming conventions



Guardrail

Don't modify certain folders



Standards

Follow specific coding standards

* Two Ways to Create CLAUDE.md

1 The /init command

```
$ cd your-project
$ claude
> /init
```

Claude scans the project and generates a CLAUDE.md containing info + instructions that help it understand the codebase, follow rules, and even flag violations during reviews.

2 Create it manually

```
# Project
See @README for overview.

# Additional Instructions
- Use Playwright for UI automation
- Use the Page Object Model
- Run pnpm test before committing
```

Just a plain Markdown file at the project root — write whatever the project needs.

🔧 BEST PRACTICE

Keep the main CLAUDE.md concise — a good rule of thumb is **under ~200 lines**. Split large rule sets into separate files instead of one giant document.

🔗 Imports with @

CLAUDE.md can **import** another file so you don't cram everything into one place:

```
# Additional Instructions
- git workflow @docs/git-instructions.md
```

CLAUDE.md

@docs/git-instructions.md

docs/git-instructions.md

@docs/deeper-rules.md (nested import)

deeper-rules.md

Imports are EXPANDED when CLAUDE.md loads — not lazy.

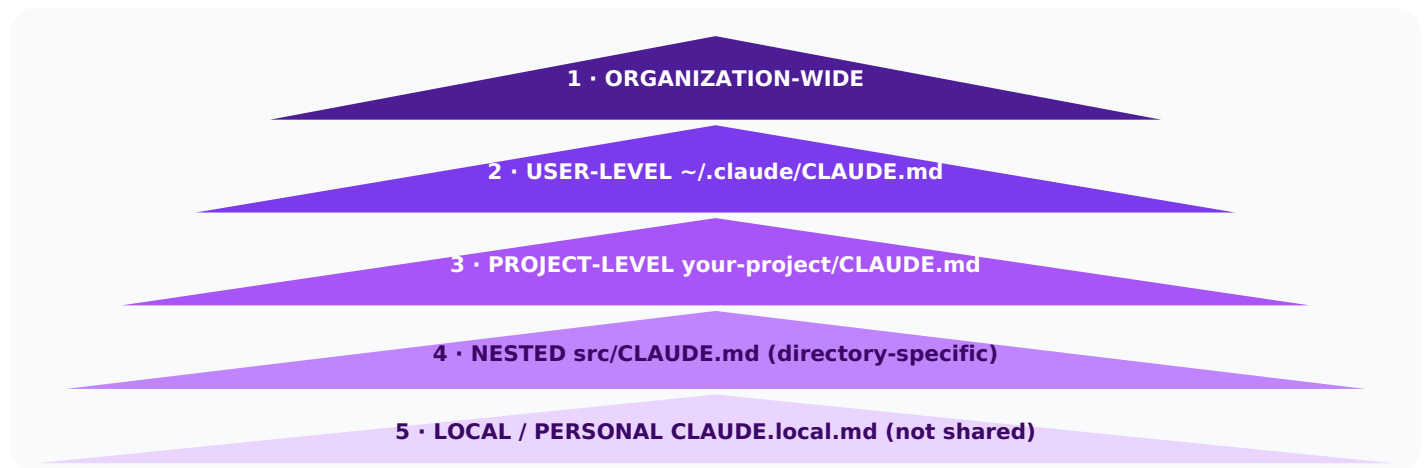
Imports can nest — content still counts toward context!

⚠ COMMON MISCONCEPTION

Splitting one large file into five smaller imported files does **not** save context — Claude still needs all the imported content loaded. Imports are purely an **organisation** mechanism, not lazy-loading.

The 5 Levels of CLAUDE.md

From company-wide rules down to your own private project notes



LEVEL	LOCATION	SCOPE	SHARED WITH TEAM?
Organization	<code>/etc/claude-code/managed-settings.json</code>	Every dev, every project on the machine	Managed centrally by IT/DevOps
User	<code>~/.claude/CLAUDE.md</code>	Just you, across ALL your projects (A, B, C...)	✗ Personal only
Project	<code>your-project/CLAUDE.md</code>	Everyone working on this one repo	✓ Committed to Git
Nested	<code>your-project/src/CLAUDE.md</code>	Only files under that sub-directory	✓ Committed to Git
Local	<code>your-project/CLAUDE.local.md</code>	Just you, this one project	✗ Not shared (like personal notes)

■ Organization example

- Don't expose sensitive information
- Follow company security guidelines
- Follow compliance requirements

🔍 User-level example

- Prefer pnpm over npm
 - Always write Playwright tests using the Page Object Model
- Personal preference, applies in Project A, B, and C alike.

🔗 ALSO GOOD TO KNOW

At the user level you can split preferences into `~/.claude/rules/preferences.md` (how you like code written) and `~/.claude/rules/workflows.md` (how you like to work), instead of one giant personal CLAUDE.md.

Full Folder Structure Cheat-Map

Where every type of CLAUDE.md-related file actually lives

```
/etc/claude-code/managed-settings.json  ← org-wide, embedded config (not a .md file)
~/.claude/
├─ CLAUDE.md                          ← USER level: applies to you, every project
├─ rules/
│  ├── preferences.md                 ← how you like code written
│  └─ workflows.md                   ← how you like to work

your-project/                          ← PROJECT root
├─ CLAUDE.md                          ← main project instructions (commit to Git)
├─ CLAUDE.local.md                   ← personal notes, THIS project only, not shared
├─ AGENTS.md                         ← shared w/ other AI tools (Cursor, etc.)
├─ .claude/
│  ├── CLAUDE.md                     ← alt. location for main project file
│  └─ rules/
│     ├── code-style.md              ← unconditional (no frontmatter)
│     ├── testing.md                 ← path-specific! paths: tests/**/*.ts
│     └─ security.md
├─ docs/
│  └─ git-instructions.md            ← imported via @docs/git-instructions.md
└─ src/
   └─ CLAUDE.md                     ← NESTED: loaded only when working under src/
```

🔗 Path-specific rule example (frontmatter)

```
---
paths:
  - "tests/**/*.ts"
---

# testing.md content only applies to matching files
```

RULE FILE	FRONTMATTER?	WHEN IT APPLIES
code-style.md	None	Unconditional — applies generally, same as being inside CLAUDE.md directly
testing.md	paths: tests/**/*.ts	Conditional — only loaded when Claude touches a matching file, e.g. tests/login.spec.ts

🔑 WHY THIS MATTERS

In a large codebase you can keep Frontend / Backend / Testing / Infra rules separate, and only pull in the relevant ones instead of loading everything, every time.

AGENTS.md & CLAUDE.local.md

Working with other AI tools, and your own private project notes

AGENTS.md is a file already used by other AI coding agents (e.g. Cursor). Claude Code does **not** automatically treat it as CLAUDE.md — you must connect them explicitly.

Option 1 — Symlink

```
$ ln -s AGENTS.md CLAUDE.md
```

CLAUDE.md now *points to* the existing AGENTS.md — one shared file, no duplication.

Option 2 — Import it

```
@AGENTS.md

## Claude Code
Use plan mode for changes
under src/billing/.
```

Reuse shared instructions from AGENTS.md, then layer **Claude-specific** instructions underneath.

PATTERN TO REMEMBER

Shared instructions first (AGENTS.md) → then Claude-specific instructions on top. Great when a repo already supports multiple AI coding tools.

CLAUDE.local.md — your personal notes



Location

Same project root as CLAUDE.md



Visibility

NOT shared with the team (not committed / gitignored)



Scope

Just you, just this one project

Think of it as sticky notes only you can see, sitting right next to the shared CLAUDE.md.

.claude/CLAUDE.md — the alternate location

You can place the main project file at the root (`CLAUDE.md`) **or** inside `.claude/CLAUDE.md` — they serve the same purpose. You only need **one** of the two as your main project file, not both.

Case Study: AI Test Case Generator

The running example project — Azure DevOps → Claude → CSV → back to ADO



🎯 BIGGER GOAL OF THE SERIES

Not just building this tool — **learning core Claude Code concepts** (CLAUDE.md, context, imports, rules) that you can re-apply directly to a real Playwright automation project.

📄 Sample CLAUDE.md rules for this project

```

# Project
AI-powered Test Case Generator — reads PBIs/AC from Azure DevOps,
generates test cases, exports to CSV, pushes approved cases back to ADO.

# Additional Instructions
- For every acceptance criterion: create positive, negative, and edge scenarios
- Generate all test cases in BDD format (Given / When / Then)
- Use only PBI title, description, and acceptance criteria from ADO
- Do NOT invent application behaviour that isn't in the PBI
  
```

💡 Example ask → structured output

A well-formed prompt built on these rules (from the transcript) requested a login story broken into **2 Positive, 3 Negative, 2 Edge, 1 Boundary, 1 Performance** case — each with Title, Preconditions, Steps, Expected Result, Test Data, and Type.

💡 PROMPTING TIP FROM THE TRANSCRIPT

"The more concrete your acceptance criteria (limits, timeouts, exact messages), the sharper the generated test cases — Claude only builds from what the story actually states."

Cheat Sheet & Self-Test

One page to revise before your next Claude Code session

📁 File location quick reference

FILE	SCOPE	COMMITTED?
/etc/claude-code/managed-settings.json	Whole organization	Managed by IT/DevOps
~/.claude/CLAUDE.md	You, all projects	✗
~/.claude/rules/*.md	You, all projects (split by topic)	✗
CLAUDE.md (or .claude/CLAUDE.md)	Whole project, whole team	✓
CLAUDE.local.md	You, this project only	✗
AGENTS.md	Any AI coding agent (needs symlink/import for Claude)	✓
src/CLAUDE.md (nested)	Only that sub-directory	✓
.claude/rules/testing.md (+frontmatter)	Only matching file paths	✓

🧠 Quick self-test (answers upside-down... just kidding, think first!)

Q1 Does a new Claude Code session automatically know what you told Claude yesterday?	Q2 What's the difference between ~/.claude/CLAUDE.md and your-project/CLAUDE.md?	Q3 Does splitting CLAUDE.md into imported files reduce how much context Claude uses?
Q4 How do you make a path-specific rule apply only to test files?	Q5 How do you connect an existing AGENTS.md to Claude Code (2 ways)?	Q6 Where would you put a personal note that only you should see, for one project?

👉 GOLDEN RULE TO REMEMBER

CLAUDE.md ≠ memory. It's a **reloadable instruction file** that gets pulled into context at the start of a session — consistency without repetition.

▶ COMING UP NEXT IN THE SERIES

Going deeper into what to put inside CLAUDE.md, and exploring the different loading locations in more detail with live demos on the Test Case Generator project.